E-Commerce Platform Database System

Relational SQL Database Project Report

Student name	Niloufar Modirshanehchi
Student ID	GH1034020
Module	Database and Big Data
GitHub repository	https://github.com/PersianSpock/database_project_ecommerce
Video demonstration	I didn’t make the video

1. Introduction

This project designs and implements a relational SQL database for an e-commerce platform that manages customers, products, stock, orders, payments, and shipping addresses.

The main objectives of the system are to:

- store structured business data in separate but related tables;
- enforce integrity through primary keys, foreign keys, unique rules, and check constraints;
- support realistic operations such as creating orders, recording payments, updating stock, and retrieving customer order histories;
- demonstrate SQL for CRUD, joins, filtering, grouping, aggregation, and reporting.

The final solution includes 9 core tables, 3 supporting views, indexes on high-use columns, realistic sample data (made by GenAI), and a set of functional queries that answer business questions such as revenue by category, top-spending customers, low-stock products, and monthly sales performance.

2. System Design

2.1 Entities and relationships

The system contains the following core entities: customers, addresses, categories, suppliers, products, inventory, orders, order_items, and payments. Customers can store multiple addresses and place multiple orders. Products belong to one category and one supplier. Each product has one inventory record. Orders connect customers to purchased products through the order_items table, which resolves the many-to-many relationship between orders and products. Payments are stored separately so that payment details remain independent from the order header and can support realistic payment tracking.

The main cardinalities are one-to-many between customers and orders, one-to-many between customers and addresses, one-to-many between categories and products, one-to-many between suppliers and products, one-to-many between orders and order_items, and one-to-many between orders and payments. There is also a one-to-one relationship between products and inventory through a unique foreign key on product_id.

2.2 ER diagram

Figure 1 shows the final entity-relationship design used for implementation.

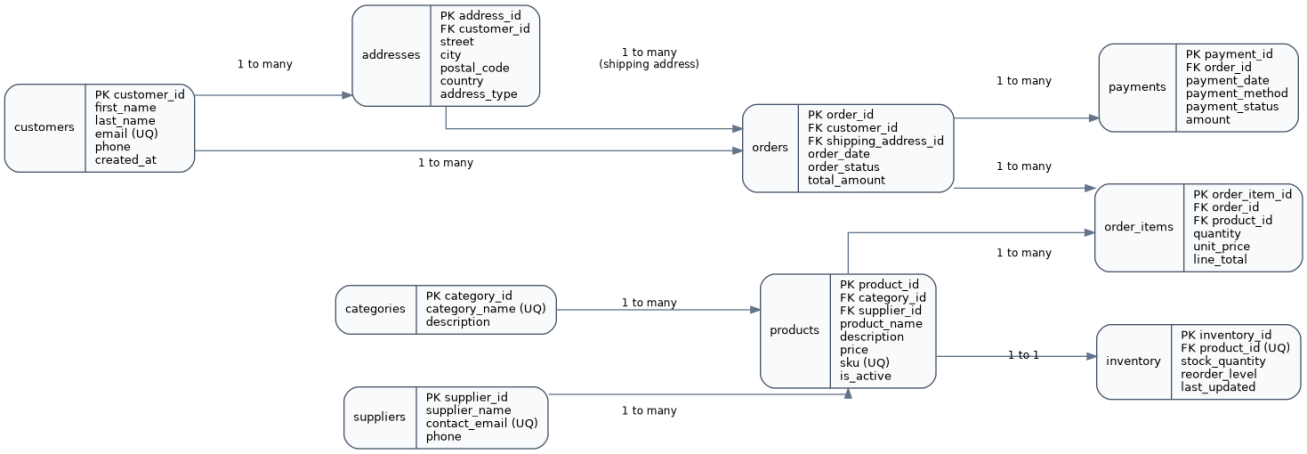

Figure 1. ER diagram for the e-commerce database

2.3 Normalization

The database was designed up to Third Normal Form (3NF):

- **1NF:** All fields contain atomic values (no repeating groups)
- **2NF:** All non-key attributes depend on the full primary key
- **3NF:** There are no transitive dependencies

For example:

- Product details are stored only in products
- Order-product relationships are handled through order_items
- Customer data is not duplicated across tables

This reduces redundancy and ensures consistency.

2.4 Constraints and data quality

To improve database quality, the design includes primary keys on every table and foreign keys to enforce referential integrity. UNIQUE constraints are used for attributes such as customer email, category name, supplier email, SKU, and the one-to-one inventory product reference. NOT NULL constraints are applied to essential fields, while CHECK constraints validate business rules such as non-negative prices, non-negative stock quantities, valid address types, valid order statuses, and valid payment statuses. Default values are also used for timestamps and status fields where appropriate.

2.5 Indexing strategy

Indexes were added to frequently joined or filtered columns: customers(email), products(category_id), products(supplier_id), orders(customer_id), orders(order_date), order_items(order_id), order_items(product_id), and payments(order_id). Although the sample dataset is modest, these indexes represent realistic design choices because they improve performance for customer lookups, reporting queries, joins, and date-based filtering in larger production-style datasets.

3. Implementation

3.1 Table Creation

All tables were created using MySQL with the InnoDB storage engine to support foreign keys and transactions.

Primary and foreign keys were defined to enforce relationships. Additional constraints were used to improve data quality:

- NOT NULL for required fields
- UNIQUE for attributes like email and SKU
- CHECK constraints for valid values (e.g., payment status)

3.2 Indexing

Indexes were added to improve query performance, especially for joins and filtering. These include:

- customers(email)
- products(category_id, supplier_id)
- orders(customer_id, order_date)
- order_items(order_id, product_id)
- payments(order_id, payment_status)

These indexes help speed up queries such as retrieving customer orders or calculating revenue.

3.3 Sample data

Disclaimer: I used GenAI for the sample data.

Sample data was inserted to simulate a real system, including:

- multiple customers
- various product categories
- different suppliers
- multiple orders and payments

This allowed meaningful queries such as identifying top customers or best-selling products.

4. Results and Queries

The system supports a range of queries that demonstrate its functionality.

Basic Queries

- Listing all products with category and supplier
- Viewing orders with customer details
- Retrieving order items with product information

Aggregation Queries

- Total revenue from completed payments
- Revenue by category
- Top 5 customers by spending
- Best-selling products

Business Logic Queries

- Products with low stock
- Orders not fully paid
- Customers with multiple orders

- Products never ordered

Advanced Queries

- Monthly sales summary using DATE_FORMAT
- Customer lifetime value with ranking
- Revenue breakdown by category
- Payment reconciliation showing outstanding balances
-

These executed queries show that the database supports both operational tasks and management reporting. Selected outputs are included below.

4.1 Orders with customer names

This join confirms that customer and order data connect correctly through foreign keys and can be displayed in a readable order history.

order_id	customer_name	order_status	total_amount
1	Aisha Rahman	delivered	132.94
2	Daniel Carter	delivered	139.97
3	Mina Patel	paid	154.0
4	Lucas Meyer	delivered	146.98
5	Grace Wilson	shipped	89.48
6	Noah Kim	delivered	178.99
7	Sofia Garcia	pending	113.49
8	Ethan Brown	cancelled	71.99
9	Priya Shah	paid	69.99
10	Leo Martin	delivered	270.97
11	Mina Patel	shipped	64.0
12	Aisha Rahman	paid	174.97

4.2 Order items with product and category details

This query proves that the many-to-many relationship between orders and products is implemented correctly through order_items.

order_id	product_name	category_name	quantity	line_total
1	Cycling Water Bottle	Sports	2	28.0
1	SQL Fundamentals	Books	1	44.95
1	Wireless Earbuds	Electronics	1	59.99
2	Air Fryer 5L	Home & Kitchen	1	89.99
2	LED Desk Lamp	Home & Kitchen	2	49.98
3	Fitness Smartwatch	Electronics	1	129.0
3	Yoga Mat	Sports	1	25.0
4	Resistance Bands Set	Sports	2	43.98
4	Running Shoes	Fashion	1	84.5
4	Sports Cap	Fashion	1	18.5
5	Smartphone Stand	Electronics	1	19.5
5	USB-C Charger 65W	Electronics	2	69.98

4.3 Revenue by category

Grouping by category shows how transaction-level data can be aggregated into business insight for product planning.

category_name	revenue
Electronics	562.44
Home & Kitchen	408.95
Sports	252.96
Books	208.43
Fashion	103.0

4.4 Top customers by spending

This result identifies high-value customers and demonstrates grouping, summation, ordering, and limiting.

customer_name	total_spent
Aisha Rahman	307.91
Leo Martin	270.97
Mina Patel	218.0
Noah Kim	178.99
Lucas Meyer	146.98

4.5 Low stock products

This query supports inventory management by showing products whose current stock has fallen below the reorder threshold.

product_name	stock_quantity	reorder_level
Dumbbell Pair 10kg	7	8
Ceramic Cookware Set	9	10
Air Fryer 5L	10	12
Yoga Mat	12	15

4.6 Monthly sales summary

Date grouping turns payment records into a time-based revenue view that could support dashboards and trend monitoring.

sales_month	monthly_revenue
2026-01	426.91
2026-02	415.45
2026-03	404.96
2026-04	174.97

Additional queries in the project include total revenue, best-selling products, orders not yet paid, customers with multiple orders, products never ordered, average order value, supplier contribution to revenue, customers with no orders, and the most popular category by units sold.

5. Challenges and Solutions

1. Trigger Limitations:

Initially, triggers were used to automatically update order totals and inventory. However, this caused errors in MySQL due to restrictions on updating tables within triggers.
Solution: Replaced trigger-based logic with explicit SQL updates in queries.

2. Data Consistency:

Ensuring that order totals matched order items required careful design.

Solution:

Recalculate totals using subqueries

Use generated columns for line_total

3. Address Ownership:

Ensuring that an order uses a valid customer address was not straightforward.

Solution: Used a composite foreign key linking orders to addresses.

6. Conclusion and Future Work

This project demonstrates a complete relational database design for an e-commerce system. It includes normalized tables, strong relationships, constraints, indexing, and meaningful queries.

The system is capable of handling realistic business operations and provides useful insights through SQL queries.

Future Improvements

- Add customer reviews
- Implement discount and coupon systems
- Add delivery tracking
- Introduce stored procedures for automation
- Expand reporting features